

条款 07: 为多态基类声明 virtual 析构函数

Declare destructors virtual in polymorphic base classes.

有许多种做法可以记录时间, 因此, 设计一个 `TimeKeeper` base class 和一些 `derived classes` 作为不同的计时方法, 相当合情合理:

```
class TimeKeeper {
public:
    TimeKeeper();
    ~TimeKeeper();
    ...
};
class AtomicClock: public TimeKeeper { ... }; //原子钟
class WaterClock: public TimeKeeper { ... }; //水钟
class WristWatch: public TimeKeeper { ... }; //腕表
```

许多客户只想在程序中使用时间, 不想操心时间如何计算等细节, 这时候我们可以设计 **factory** (工厂) 函数, 返回指针指向一个计时对象。Factory 函数会“返回一个 **base class** 指针, 指向新生成之 **derived class** 对象”:

```
TimeKeeper* getTimeKeeper();           //返回一个指针, 指向一个
                                         //TimeKeeper 派生类的动态分配对象
```

为遵守 **factory** 函数的规矩, 被 `getTimeKeeper()` 返回的对象必须位于 **heap**。因此为了避免泄漏内存和其他资源, 将 **factory** 函数返回的每一个对象适当地 **delete** 掉很重要:

```
TimeKeeper* ptk = getTimeKeeper();      //从 TimeKeeper 继承体系
                                         //获得一个动态分配对象。
...                                     //运用它...
delete ptk;                             //释放它, 避免资源泄漏。
```

条款 13 说“倚赖客户执行 **delete** 动作, 基本上便带有某种错误倾向”, 条款 18 则谈到 **factory** 函数接口该如何修改以便预防常见之客户错误, 但这些在此都是次要的, 因为此条款内我们要对付的是上述代码的一个更根本弱点: 纵使客户把每一件事都做对了, 仍然没办法知道程序如何行动。

问题出在 `getTimeKeeper` 返回的指针指向一个 **derived class** 对象 (例如 `AtomicClock`), 而那个对象却经由一个 **base class** 指针 (例如一个 `TimeKeeper*` 指针) 被删除, 而目前的 **base class** (`TimeKeeper`) 有个 **non-virtual** 析构函数。

这是一个引来灾难的秘诀, 因为 C++ 明白指出, 当 **derived class** 对象经由一个 **base class** 指针被删除, 而该 **base class** 带着一个 **non-virtual** 析构函数, 其结果未有定义——实际执行时通常发生的是对象的 **derived** 成分没被销毁。如果 `getTimeKeeper` 返回指针指向一个 `AtomicClock` 对象, 其内的 `AtomicClock` 成分 (也就是声明于 `AtomicClock` class 内的成员变量) 很可能没被销毁, 而 `AtomicClock` 的析构函数也未能执行起来。然而其 **base class** 成分 (也就是 `TimeKeeper` 这一部分) 通常会被销毁, 于是造成一个诡异的“局部销毁”对象。这可是形成资源泄漏、败坏之数据结构、在调试器上浪费许多时间的绝佳途径喔。

消除这个问题的做法很简单: 给 **base class** 一个 **virtual** 析构函数。此后删除 **derived class** 对象就会如你想要的那般。是的, 它会销毁整个对象, 包括所有 **derived class** 成分:

```
class TimeKeeper {
public:
    TimeKeeper( );
    virtual ~TimeKeeper();
    ...
};
TimeKeeper* ptk = getTimeKeeper();
...
delete ptk;                                //现在, 行为正确。
```

像 `TimeKeeper` 这样的 **base classes** 除了析构函数之外通常还有其他 **virtual** 函数, 因为 **virtual** 函数的目的是允许 **derived class** 的实现得以客制化 (见条款 34)。例如 `TimeKeeper` 就可能拥有一个 **virtual** `getCurrentTime`, 它在不同的 **derived classes** 中有不同的实现码。任何 **class** 只要带有 **virtual** 函数都几乎确定应该也有一个 **virtual** 析构函数。

如果 **class** 不含 **virtual** 函数, 通常表示它并不意图被用做一个 **base class**。当 **class** 不企图被当作 **base class**, 令其析构函数为 **virtual** 往往是个馊主意。考虑一个用来表示二维空间点坐标的 **class**:

```
class Point {                                //一个二维空间点 (2D point)
public:
    Point(int xCoord, int yCoord);
    ~Point();
private:
    int x, y;
};
```

如果 `int` 占用 32 bits，那么 `Point` 对象可塞入一个 64-bit 缓存器中。更有甚者，这样一个 `Point` 对象可被当做一个“64-bit 量”传给以其他语言如 C 或 FORTRAN 撰写的函数。然而当 `Point` 的析构函数是 `virtual`，形势起了变化。

欲实现出 `virtual` 函数，对象必须携带某些信息，主要用来在运行期决定哪一个 `virtual` 函数该被调用。这份信息通常是由一个所谓 `vp`tr (virtual table pointer) 指针指出。`vp`tr 指向一个由函数指针构成的数组，称为 `vtbl` (virtual table)；每一个带有 `virtual` 函数的 `class` 都有一个相应的 `vtbl`。当对象调用某一 `virtual` 函数，实际被调用的函数取决于该对象的 `vp`tr 所指的那个 `vtbl`——编译器在其中寻找适当的函数指针。

`virtual` 函数的实现细节不重要。重要的是如果 `Point` `class` 内含 `virtual` 函数，其对象的体积会增加：在 32-bit 计算机体系结构中将占用 64 bits（为了存放两个 `ints`）至 96 bits（两个 `ints` 加上 `vp`tr）；在 64-bit 计算机体系结构中可能占用 64~128 bits，因为指针在这样的计算机结构中占 64 bits。因此，为 `Point` 添加一个 `vp`tr 会增加其对象大小达 50%~100%！`Point` 对象不再能够塞入一个 64-bit 缓存器，而 C++ 的 `Point` 对象也不再和其他语言（如 C）内的相同声明有着一样的结构（因为其他语言的对应物并没有 `vp`tr），因此也就不再可能把它传递至（或接受自）其他语言所写的函数，除非你明确补偿 `vp`tr——那属于实现细节，也因此不再具有移植性。

因此，无端地将所有 `classes` 的析构函数声明为 `virtual`，就像从未声明它们为 `virtual` 一样，都是错误的。许多人的心得是：只有当 `class` 内含至少一个 `virtual` 函数，才为它声明 `virtual` 析构函数。

即使 `class` 完全不带 `virtual` 函数，被“non-virtual 析构函数问题”给咬伤还是有可能的。举个例子，标准 `string` 不含任何 `virtual` 函数，但有时候程序员会错误地把它当做 `base class`：

```
class SpecialString: public std::string { //傻主意! std::string 有个
    ...                                //non-virtual 析构函数
};
```

乍看似似乎无害，但如果你在程序任意某处无意间将一个 `pointer-to-SpecialString`

转换为一个 pointer-to-string, 然后将转换所得的那个 string 指针 delete 掉, 你立刻被流放到“行为不明确”的恶地上:

```
SpecialString* pss = new SpecialString("Impending Doom");
std::string* ps;
...
ps = pss;                //SpecialString* => std::string*
...
delete ps;               //未有定义! 现实中*ps 的 SpecialString 资源会泄漏,
                        //因为 SpecialString 析构函数没被调用。
```

相同的分析适用于任何不带 virtual 析构函数的 class, 包括所有 STL 容器如 vector, list, set, tr1::unordered_map (见条款 54) 等等。如果你曾经企图继承一个标准容器或任何其他“带有 non-virtual 析构函数”的 class, 拒绝诱惑吧! (很不幸 C++ 没有提供类似 Java 的 final classes 或 C# 的 sealed classes 那样的“禁止派生”机制。)

有时候令 class 带一个 pure virtual 析构函数, 可能颇为便利。还记得吗, pure virtual 函数导致 abstract (抽象) classes —— 也就是不能被实体化 (instantiated) 的 class。也就是说, 你不能为那种类型创建对象。然而有时候你希望拥有抽象 class, 但手上没有任何 pure virtual 函数, 怎么办? 唔, 由于抽象 class 总是企图被当作一个 base class 来用, 而又由于 base class 应该有个 virtual 析构函数, 并且由于 pure virtual 函数会导致抽象 class, 因此解法很简单: 为你希望它成为抽象的那个 class 声明一个 pure virtual 析构函数。下面是个例子:

```
class AWOV {                                //AWOV = "Abstract w/o Virtuals"
public:
    virtual ~AWOV() = 0;                    //声明 pure virtual 析构函数
};
```

这个 class 有一个 pure virtual 函数, 所以它是个抽象 class, 又由于它有个 virtual 析构函数, 所以你不需担心析构函数的问题。然而这里有个窍门: 你必须为这个 pure virtual 析构函数提供一份定义:

```
AWOV::~~AWOV() { }                        //pure virtual 析构函数的定义
```

析构函数的运作方式是, 最深层派生 (most derived) 的那个 class 其析构函数最先被调用, 然后是其每一个 base class 的析构函数被调用。编译器会在 AWOV 的 derived

classes 的析构函数中创建一个对~AWOV 的调用动作，所以你必须为这个函数提供一份定义。如果不这样做，连接器会发出抱怨。

“给 base classes 一个 virtual 析构函数”，这个规则只适用于 polymorphic（带多态性质的）base classes 身上。这种 base classes 的设计目的是为了用来“通过 base class 接口处理 derived class 对象”。TimeKeeper 就是一个 polymorphic base class，因为我们希望处理 AtomicClock 和 WaterClock 对象，纵使我们只有 TimeKeeper 指针指向它们。

并非所有 base classes 的设计目的都是为了多态用途。例如标准 string 和 STL 容器都不被设计作为 base classes 使用，更别提多态了。某些 classes 的设计目的是作为 base classes 使用，但不是为了多态用途。这样的 classes 如条款 6 的 Uncopyable 和标准程序库的 input_iterator_tag（条款 47），它们并非被设计用来“经由 base class 接口处置 derived class 对象”，因此它们不需要 virtual 析构函数。

请记住

- polymorphic（带多态性质的）base classes 应该声明一个 virtual 析构函数。如果 class 带有任何 virtual 函数，它就应该拥有一个 virtual 析构函数。
- Classes 的设计目的如果不是作为 base classes 使用，或不是为了具备多态性（polymorphically），就不该声明 virtual 析构函数。

条款 08：别让异常逃离析构函数

Prevent exceptions from leaving destructors.

C++ 并不禁止析构函数吐出异常，但它不鼓励你这样做。这是有理由的。考虑以下代码：

```
class Widget {
public:
    ...
    ~Widget() { ... }           //假设这个可能吐出一个异常
};
void doSomething()
{
    std::vector<Widget> v;
    ...
}                               //v 在这里被自动销毁
```

当 `vector v` 被销毁, 它有责任销毁其内含的所有 `widgets`。假设 `v` 内含十个 `widgets`, 而在析构第一个元素期间, 有个异常被抛出。其他九个 `widgets` 还是应该被销毁 (否则它们保存的任何资源都会发生泄漏), 因此 `v` 应该调用它们各个析构函数。但假设在那些调用期间, 第二个 `widget` 析构函数又抛出异常。现在有两个同时作用的异常, 这对 C++ 而言太多了。在两个异常同时存在的情况下, 程序若不是结束执行就是导致不明确行为。本例中它会导致不明确的行为。使用标准程序库的任何其他容器 (如 `list`, `set`) 或 TR1 的任何容器 (见条款 54) 或甚至 `array`, 也会出现相同情况。容器或 `array` 并非遇上麻烦的必要条件, 只要析构函数吐出异常, 即使并非使用容器或 `arrays`, 程序也可能过早结束或出现不明确行为。是的, C++ 不喜欢析构函数吐出异常!

这很容易理解, 但如果你的析构函数必须执行一个动作, 而该动作可能会在失败时抛出异常, 该怎么办? 举个例子, 假设你使用一个 `class` 负责数据库连接:

```
class DBConnection {
public:
    ...
    static DBConnection create();           //这个函数返回
                                           // DBConnection 对象;
                                           //为求简化暂略参数。
    void close();                           //关闭联机; 失败则抛出异常。
};
```

为确保客户不忘记在 `DBConnection` 对象身上调用 `close()`, 一个合理的想法是创建一个用来管理 `DBConnection` 资源的 `class`, 并在其析构函数中调用 `close`。这一类用于资源管理的 `classes` 在第 3 章有详细探讨, 这儿只要考虑它们的析构函数长相就够了:

```
class DBConn {                               //这个 class 用来管理 DBConnection 对象
public:
    ...
    ~DBConn()                               //确保数据库连接总是会被关闭
    {
        db.close();
    }
private:
    DBConnection db;
};
```

这便允许客户写出这样的代码: